Exercises week 9

Last update 2026-10-22

These exercises aim to give you insight into and practical experience with the performance of parallel code on multi-core shared-memory computers:

- platform threads, virtual threads and the executor framework;
- algorithms exploiting multiple threads to improve performance of e.g. sorting;
- the impact of memory and especially cache coherence on the performance of concurrent data structures.

Do this first

The exercises build on the lecture, the lecture note chapters *8. The computer's CPU and RAM* and *9. Scalability case studies* including section *9.1 Platform threads, virtual threads and futures* and Goetz et al. 5, 6.1, 6.2 and 11.

All the relevant Java code can be found on the Github page for week 9: (https://github.itu.dk/PCPP/PCPP2026-Public/tree/main/week09). The code for the exercises is in:

`.../week09/code-exercises/week09exercises/....`

Mandatory

Exercise 9.1 This exercise builds on the parallel prime counting examples with platform and virtual threads (`TestCountPrimesThreads.java`) and with tasks and executors (`TestCountPrimesTasks.java`).

If the measurements take excessively long time on your computer, you may measure for `range` being `100_000` instead of `1_000_000`.

1. Measure the performance of parallel prime counting using platform threads, on your own hardware, as a function of the number of threads that share the work of counting primes. Record system information as well as the measurement results for 1, 2, 3, ..., 8, 16, 32, 64, ..., 2048 platform threads in a text file.
2. Make the same measurements for virtual threads.
3. Make the same measurement for the executor-and-task version, still for 1, 2, 3, ..., 8, 16, 32, 64, ..., 2048 parallel tasks.
4. Reflect and comment on the results; are they plausible? Is there any reasonable relation between the number of threads that gave best performance, and the number of cores in the computer you ran the benchmarks on? Any surprises?

   Challenging

5. Now change the worker thread code in so that it performs the counting locally in the thread. That is, create a local variable `int count = 0` inside the lambda `tb.start(() -> { ... })` that defines the thread's task. Inside the for-loop, increment `count` instead of calling `lc.increment()`. The local variable is thread-confined and updating it needs no synchronization. After the for-loop, add `count` to the shared LongCounter.

   This reduces the number of synchronizations enormously and in theory should make the code faster. Measure whether this is the case on your hardware. Hint: You may choose to do this just for `threadCount = 16`, otherwise the measurements take a lot of time.

Mandatory

Exercise 9.2 The goal of this exercise is to build various threadsafe histogram classes, in continuation of exercise 6.1. Your implementations should implement the same interface, in `HistogramPerformance.java`:

```
interface Histogram {
  void increment(int bin);
  int getCount(int bin);
  int getSpan();
  int getAndClear(int bin);
}
```

File `HistogramPerformance.java` contains a parallel workload for thread-safe classes implementing this interface, also reporting the time spent executing the workload. You can measure the performance of a histogram object by passing it to static method `exerciseHistogram`. For instance:

```
exerciseHistogram(new SynchronizedHistogram(30));
```

As before, it is enough for the histogram to have a span of 30 integer bins.

In the questions below you are asked to implement and measure various threadsafe and scalable histogram classes, borrowing ideas from the long adder and hash map case studies.

1. Based on the non-threadsafe implementation in `Histogram1.java`, make a threadsafe SynchronizedHistogram class that uses a single lock. The class should use a single `int[]` for the counts. Run it with the parallel workload described above. Note down the time it takes.
2. Take your CasHistogram implementation from week 06, run it with the parallel workload described above, and note down the time it takes. Is it faster or slower than the single-lock implementation?
3. Make an AtomicHistogram class that uses an `AtomicIntegerArray` for the counts and relevant methods such as `get` and `getAndIncrement`. Run it with the parallel workload and note the time. How does it compare to the previous versions?
4. Make a StripedHistogram class in that uses an `AtomicIntegerArray[]` for the counts, where the length of the outer array is 16, equal to the number of stripes. The histogram's `increment(bin)` can be implemented like this:

   ```
   int strp = (int)(Thread.currentThread().threadId() % stripeCount);
   counts[strp].getAndIncrement(bin);
   ```

   The idea is that the counts of a bin get distributed on 16 stripes, where each stripe is represented by an AtomicIntegerArray. A stripe corresponds to a group of calling threads, and the stripe index `strp` is computed from the thread's id as shown above. Implementing `getCount` and `getAndClear` now requires summing over the 16 relevant components. Run it with the parallel workload and note the time. How does it compare to the previous versions?
5. Which of the long adder or hash map implementations (there may be more than one of them) does the most StripedHistogram resemble? (Don't judge from the name alone; look at the implementations).
6. Make a LongAdderHistogram class that uses a `LongAdder[]` for the counts, where LongAdder is the Java library class. Now `increment(bin)` is implemented simply by `counts[bin].increment()`. Run it with the parallel workload and note the time. How does it compare to the previous versions? Could there be drawbacks to using an array of LongAdder objects instead of the previous solutions?

   Challenging
7. Choose some other and potentially better threadsafe histogram design, implement, describe and measure it, and hand in the implementation, description, results and brief discussion of what you expected and how it actually performed.

Mandatory

Exercise 9.3 File `LongAdderPerformance.java` contains several implementations of a long (64-bit) integer with thread-safe `add` and `longValue` operations, as explained in the lecture and in the lecture notes section *9.2 Long adder scalability case study*.

1. Edit the file to make the performance measurements only for `threadCount` being 20, otherwise they run for a long time. Compile and run the code to measure, on your own hardware, the performance of the various atomic long implementations.

   Report the platform information and the numbers and discuss whether they are plausible, e.g. relative to the number of cores in your machine.

2. Explain in your own words why the SyncLongArrayAdderPadded implementation is faster than the SyncLongArrayAdder implementation (if indeed this is the case on your machine).

3. In the SyncLongArrayAdderPadded class, both the `counts` and `locks` arrays are padded with 15 unused array elements between every array element that is actually used. The elements that are used have indexes 0, 16, 32, ... because `padding` is 16.

   Moreover, the `locks` array is filled with new objects created by `new Object()`, so that their intrinsic locks can be used for mutual exclusion. However, only the objects in `locks[0]`, `locks[16]`, `locks[32]`, ... are actually ever used, so it may seem wasteful to create all the other objects.

   Your task: Create a new class SyncLongArrayAdderLessPad as a variant of SyncLongArrayAdderPadded where the constructor allocates only those objects that are needed for locking. That is, inside the for-loop, a new object should be created and assigned to `locks[s]` only when `s % padding == 0`, that is, when `s` is a multiple of 16. Otherwise, `locks[s]` should be left with its initial value `null`, which is safe because those array elements will never be referred to.

   Measure the performance of this new class along with SyncLongAdder, SyncLongArrayAdder and SyncLongArrayAdderPadded (no need to measure the others again).

   Report the measurement results. Is there any drastic performance difference between the (apparently wasteful) SyncLongArrayAdderPadded class and the SyncLongArrayAdderLessPad class?

   If there is a drastic performance difference, how could it be explained?

Mandatory

Exercise 9.4 Run the memory latency measurement on your own machine by uncommenting the line below in the `main` method of file `Benchmark.java`:

```
MemoryLatency.MemoryLatencies();
```

This should give an impression of the cache memory hierarchy of your computer's CPU. Report the numbers and discuss how they compare to those shown in figure 9 in section 10 of the note `Microbenchmarks in Java, C# and C`. How large is the level 1 data cache in your CPU, judging from these numbers? How fast is access to the RAM?